Ders 8 Çalışma Özeti

Ders 8 Çalışma Özeti

Genel Konular

- Refactoring kavramı (ekrandan, 00:06:00)
  - Bir kod parçasının ne yaptığını değiştirmeden nasıl yaptığını değiştirmektir. Daha dar tanımı: yazılımın dışarıdan görünen davranışı değiştirilmeden iç yapısının değiştirilmesi.
  - "Dışarıdan bakan" yalnız insan değildir; sistemle tümleşik çalışan başka yazılımlar da olabilir. Bu durumda dışarıya açılan erişim arayüzü (sınıf adları, sınıf ve metot imzaları) de korunması gereken davranışın parçasıdır.
  - Amaç kodun iç kalite özelliklerini iyileştirmektir. İç kalite özellikleri programcıyı ve yazılım ekibini, dış kalite özellikleri son kullanıcıyı ilgilendirir.
  - Slayttaki amaçlar: anlaşılabilirliğin artırılması, koda esneklik kazandırılması, yeniden kullanılabilirliğin artırılması, yüksek uyum ve düşük bağlaşımın sağlanması.
- Refactoring nasıl ve ne zaman yapılır (ekrandan, 00:10:00)
  - Çevik yaklaşımla yapılır: mevcut gereksinimleri karşılayan ilk tasarım yapılır, bu tasarımın mükemmel olması ve her bilinmezi öngörmesi gerekmez, yeterince iyi tasarım gerçeklenmeye başlanır, önceki bir tasarım kararının yanlış olduğu anlaşılınca refactoring yapılır.
  - Sürekli sınama ile yapılır: değişikliklerin yeni hata getirmediğinden emin olmak için her değişikliğin ardından kod yeniden sınanır. Kaç küçük adımdan sonra kapsamlı sınama (regresyon sınaması) yapılacağına geliştirici karar verir; hoca bir sorumluluk tamamlandığında sınamayı öneriyor.
  - Ne zaman: yeni özellik eklemekte zorlanmaya başlandığında, bir hatayı bulmak zorlaştığında ya da düzeltmek için birçok ayrı yeri kurcalamak gerektiğinde. Kısaca "kod kötü kokmaya başladığında".
- Code smell (kötü kokan / kusurlu kod) (ekrandan, 00:14:20)
  - Yazılan kodu iyileştirme gerektiğinin işaretleridir; kod içindeki tersliklerdir; kullanılan programlama yaklaşımının kusurlu kullanım örnekleridir.
  - Hata (bug) değildir: program doğru çalışır ama verimsiz çalışmaya ya da değişiklik yapmayı zorlaştırmaya yol açar. Hoca bu yüzden Türkçe karşılık olarak "kusurlu kod" demeyi tercih ediyor.
  - Zamanla yazılım "yamalı bohçaya" dönüşebilir: değişik zamanlarda farklı kişiler kodun değişik yerlerini değiştirir; bu değişiklikler birlikte planlanabilseydi daha iyi bir çözüme ulaşılabilirdi.
  - Terslikleri düzeltmek için yeniden yapılandırma eylemleri yürütülür. Bir eylem yeni kokular çıkarabilir; o zaman yeni eylemler yürütülür. Refactoring bütüncül bir yeniden tasarım değil, küçük adımlarla yapılan değişikliklerdir.
- Birim sınaması (unit testing) (ekrandan, 00:20:00)
  - En küçük yazılım bileşeninin sınanmasıdır; nesneye dayalı programlamada bireysel sınıfların sınanmasıdır.
  - Ne zaman tasarlanır: kodlamadan önce (Test Driven Development), kodlama sırasında ya da kodlamanın ardından. Önemli olan, sınamayı her şey bittikten sonraya (şelale modelindeki gibi) bırakmamaktır.
  - Bir sınıfın tek başına yürütemediği sorumluluklar için, henüz yazılmamış iş birlikçi sınıfların yerine geçen kod yazılabilir: vekil, sahte, yalancı kod/sınıf (stub, dummy, surrogate, proxy). Vekil kod yalnız gerçek sınıflar gerçeklenene dek kullanılır.
- JUnit (ekrandan, 00:20:00)

  - Java ile yazılmış kodun birim sınamaları için bir çerçeve programdır (framework). Başka diller için de sürümleri vardır (C# için csUnit).
  - Eclipse ve NetBeans ile hazır gelir (Eclipse'te build path → add libraries). İstenirse güncel sürüm indirilip projeye eklenebilir.
- JUnit 3.x ile test case hazırlamak (ekrandan, 00:30:00)
  - `junit.framework` paketinde iki temel sınıf vardır: `TestCase` (`# setup()`) ve onun kullandığı `Assert`.
  - Her uygulama sınıfı (AppClass1, AppClass2) için `TestCase`'ten kalıtımla bir test sınıfı (TestAppClass1, TestAppClass2) yazılır. Test metotlarının adı `test` ile başlar (`testXXX()`).
  - Assert metotları: `assertEquals(beklenen, sonuc : Object)`, `assertNotEquals(Object, Object)`, `assertTrue(boolean)`, `assertFalse(boolean)`, `assertNull(Object)`, `assertNotNull(Object)`.
  - `setup` metodu her `testXXX()` metodundan önce yeniden çalışır; bütün testlerden önce bir kez değil.
  - Test case'ler bir test suite altında toplanabilir. `TestAll` sınıfını yazmak zorunlu değildir; `TestAll.suite()` statiktir.
- JUnit 4.x gelişmeleri (ekrandan, 00:31:50)
  - Geriye doğru uyumluluk korunur; annotation desteği gelir (büyük/küçük harf duyarlıdır).
  - Test sınıflarının `TestCase`'ten kalıtımla türetilmesi gerekmez, ama `import static org.junit.Assert.*;` ve `import org.junit.*;` eklenmelidir.
  - Test metotlarının adı `test` ile başlamak zorunda değildir; başına `@Test` koymak yeter. `setUp` yerine istenen adlı bir metot `@Before` ile işaretlenebilir.
  - Atılması gereken bir exception'ın gerçekten atılıp atılmadığı `@Test(expected=SomeException.clas` ile sınanabilir; checked exception ise metot imzasında `throws SomeException` beyan edilir.
- Video kiralama örneği (Fowler'ın Refactoring kitabından) (ekrandan, 00:40:00)
  - Üç sınıf: `Movie (-_priceCode : int)`, `Rental (-_daysRented : int)`, `Customer (+statement())`. Customer 1 → * Rental, Rental * → 1 Movie.
  - Customer ile Movie arasına Rental konmasının nedeni: kiralama süresi Movie'de tutulursa aynı filmi başka biri başka süreyle kiraladığında bilgi bozulur; kim neyi ne kadar kiraladı geçmişi ayrı bir sınıfta tutulmalıdır.
  - Üç film türü vardır: normal (REGULAR = 0), yeni çıkan (NEW_RELEASE = 1), çocuk (CHILDRENS = 2). Her türün ücreti ve kiralama süresi farklıdır.
  - Sıralama şemasında `statement()` her kira kaydı için getMovie, getPriceCode, getDaysRented ile ücreti; tekrar getPriceCode ve getDaysRented ile bonusu hesaplar, getTitle ile makbuz satırını basar; biriken bilgiyle toplam borç ve toplam puan yazılır.
  - Kodun ilk hali `refactoring/fowler_00` paketindedir; her refactoring eyleminin sonucu yeni bir pakette (fowler_01, fowler_02, …) tutulur. Test kodları `testing` kaynak klasöründe aynı adlı paketlerdedir.
- statement() metodunun ilk hali (ekrandan, 00:50:00 ve 00:51:45)
  - Ücretler: REGULAR 2.0 (2 günü aşan her gün için +1.5), NEW_RELEASE gün başına 3.0, CHILDRENS 1.5 (3 günü aşan her gün için +1.5).
  - Her kiralama 1 puan kazandırır; yeni çıkan film 1 günden uzun kiralanırsa 1 bonus puan daha.
  - Metot 17. satırdan 56. satıra kadar uzanır. Hocaya göre en bariz iki kusur metodun uzunluğu ve switch-case yapısıdır.
- Örneğin JUnit testleri (ekrandan, 00:57:20, 01:00:00, 01:02:55, 01:04:10, 01:04:45, 01:05:25)
  - `TestMovie.testMovie`: üç türden film oluşturulur, fiyat kodlarının farklı olduğu `assertTrue` / `assertFalse` ile denetlenir.
  - `TestCustomer.setUp`: bir müşteri ve dört film (The Matrix normal, Monsters, Inc. çocuk, Surrogates ve Terminator Salvation yeni) oluşturulur; kiralama setUp'ta yapılmaz, her testte ayrı yapılır.
  - Testler: `testGetName`, `testStatementWhenEmpty`, `testStatementWithMoviesLongRent`, `testStatementWithMoviesShortRent`, `testNewReleaseRentalBonus`. Her biri elle hesaplanan beklenen makbuzu `assertEquals` ile karşılaştırır.
  - `TestAll.suite()`: `TestSuite` nesnesi oluşturur, `addTestSuite(TestCustomer.class)` ve `addTestSuite(TestMovie.class)` ile test sınıflarını ekler (ekrandan, 01:10:00).

Hocanın Özellikle Vurguladığı Kısımlar

- Refactoring mutlaka sürekli sınama ile yapılmalı
  - Refactoring eylemleri sisteme istemeden hata sokabilir. Bu yüzden her anlamlı değişiklikten, en geç bir sorumluluk tamamlanınca, sınama yapılmalıdır.
- Code smell hata değildir
  - Program işini yapar ama en iyi şekilde yapmaz. Kusur, bakım masrafını artıran bir tehlike işaretidir.
- Hataların erken teşhisi
  - Geliştirme süreci ilerledikçe değişiklik ve düzeltme masrafı üstel olarak artar. Sınama en sona bırakılmamalıdır.
- setUp her test metodundan önce yeniden çalışır
  - Bütün testlerden önce bir kez çalıştığı sanılırsa test kodu yanlış yazılabilir. Başarısız bir test bazen uygulama kodundaki değil, test kodundaki hatayı gösterir.
- Test metodu adları açıklayıcı olmalı
  - `test1`, `test2` gibi adlar değil, neyin hangi koşulda sınandığını anlatan uzun adlar (`testStatementWhenEmpty`, `testStatementWithMoviesLongRent`) kullanılmalı. JUnit 4'te zorunlu olmasa da hoca test sınıf ve metot adlarını `test` ile başlatmayı öneriyor.
- Sequence diyagramında geri dönüş okları
  - Geri dönüş okları çizilmezse bir metot çağrısının öncekinin bitiminden sonra mı yoksa onunla paralel mi çalıştığı anlaşılmaz. Hoca bunu bu yüzden önemsiyor.
- Üye alanların alt çizgiyle adlandırılması
  - Fowler örnekte üye alanları `_name`, `_rentals` gibi adlandırmış. IDE renklendirmesinin olmadığı basılı ortamda (kitap, ders notu) üye alanı metot parametresinden ayırt etmeyi kolaylaştırır.
- Test kodunu fiziksel olarak ayrı, mantıksal olarak aynı pakette tutmak
  - Test kodu ayrı bir kaynak klasöründe ama uygulama koduyla aynı adlı pakette durur. Böylece test kodu uygulama kodunu import etmek zorunda kalmaz ve yanlış sürümü import etme riski ortadan kalkar.

Kısa Tekrar Notları

- Refactoring: ne yapıldığını değiştirmeden nasıl yapıldığını değiştirmek; dışarıdan görünen davranış korunur.
- Amaç iç kaliteyi artırmak: anlaşılabilirlik, esneklik, yeniden kullanılabilirlik, yüksek uyum, düşük bağlaşım.
- Çevik yaklaşımla ve sürekli sınamayla, küçük adımlarla yapılır.
- Code smell: kodu iyileştirme gereğinin işareti; hata değil, kusur.
- Bir refactoring eylemi yeni kokular çıkarabilir; o zaman yeni eylem yapılır.
- Birim sınaması: NYP'de bireysel sınıfın sınanması. TDD, kodlama sırasında ya da sonrasında tasarlanabilir.
- Henüz yazılmamış iş birlikçi sınıfların yerine stub/dummy/surrogate/proxy kod kullanılabilir.
- JUnit 3: `TestCase`'ten kalıt, metotları `test` ile başlat, `setUp` her testten önce çalışır, `TestAll.suite()` statik.
- JUnit 4: kalıtım gerekmez; `@Test`, `@Before`, `@Test(expected=...)`.
- Video kiralama: Movie, Rental, Customer; uzun `statement()` ve switch-case ilk kusurlardır.
- Test başarısız olunca JUnit kırmızı çubuk gösterir, beklenen ve elde edilen metnin ayrıldığı yeri köşeli parantezle işaretler.

Detaylı Açıklamalar

- Refactoring'in yeri
  - Bakım, yazılım sahaya çıktıktan sonra yapılan her türlü değişikliktir: hata düzeltme, gereksinim değişikliği, yeni gereksinim. Hocaya göre bir yazılım projesindeki emeğin yaklaşık yarısı bakım aşamasında harcanır; kodun kolay anlaşılması bakım masrafını düşürür.

  - Refactoring yalnız bakım aşamasına özgü değildir. Çevik geliştirmede her döngünün yeni bir özellik (increment) eklemesi gerekmez; bir döngü kodu toparlamaya ayrılabilir.
  - Yeniden kullanılabilirlik amaçlı refactoring sahadaki kod üzerinde değil, ekibin kendi kod kitaplığının bir sürümü üzerinde yapılabilir; bu sürüm sonra başka projelere aktarılabilir.
  - Hoca "refactoring" için uygun bir Türkçe karşılık bulamadığını söyledi; "yeniden yapılandırma" ifadesi finansal anlamda borç yapılandırmayı da çağrıştırıyor.

- JUnit 3 ile çalışma biçimi
  - Kara kutu sınamasında iç çalışmanın ayrıntısı bilinmeden, verilen girdiye karşılık doğru çıktının üretilip üretilmediğine bakılır. Assert metotları bu senaryoya göre çalışır.
  - Bir test metodunda tek assert önerilir ama birden fazla olabilir; o zaman test metodunun başarılı sayılması için hepsinin geçmesi gerekir.
  - Assert metotlarının ilk parametresi isteğe bağlı bir String olabilir: test başarısız olursa ekrana yazılacak açıklama (örnek: `"Bunlar farklı tür filmlerdir"`).
  - Test sınıfına sağ tıklayıp "Run As → JUnit Test" denince bütün test metotları sırayla çalışır. Yeşil çubuk bütün testlerin geçtiğini, kırmızı çubuk en az bir testin başarısız olduğunu gösterir.

- Video kiralama kodu (ekrandan, 00:48:40, 00:50:00, 00:51:45, 00:52:40)

```
public class Customer {
    private String _name;
    private Vector<Rental> _rentals = new Vector<Rental>();
    public Customer (String name){ _name = name; }
    public void addRental(Rental arg) { _rentals.addElement(arg); }
    public String getName (){ return _name; }
    public String statement( ) {
        double totalAmount = 0;
        int frequentRenterPoints = 0;
        Enumeration<Rental> rentals = _rentals.elements();
        String result = "Rental Record for " + ...
        while (rentals.hasMoreElements()) {
            double thisAmount = 0;
            Rental each = rentals.nextElement();
            //determine amounts for each line
            switch (each.getMovie().getPriceCode()) {
            case Movie.REGULAR:
                thisAmount += 2.0;
                if (each.getDaysRented() > 2)
                    thisAmount += (each.getDaysRented() - 2) * 1.5;
                break;
            case Movie.NEW_RELEASE:
                thisAmount += each.getDaysRented() * 3.0;
                break;
            case Movie.CHILDRENS:
                thisAmount += 1.5;
                if (each.getDaysRented() > 3)
                    thisAmount += (each.getDaysRented() - 3) * 1.5;
                break;
            }
            // add frequent renter points
            frequentRenterPoints ++;
            // add bonus for a two day new release rental
            if ((each.getMovie().getPriceCode() == Movie.NEW_RELEASE)
                    && each.getDaysRented() > 1) frequentRenterPoints ++;
            //show figures for this rental
            result += "\t" + each.getMovie().getTitle()+ "\t" +
            String.valueOf(thisAmount) + "\n";
```

```
            totalAmount += thisAmount;
        }
        ...
    }
}
```

- Kod parçadır, derlenmedi. Satır 21'in sonu ve metodun son satırları ekranda görünmüyordu (`...` ile gösterildi). Testlerdeki beklenen metne göre makbuz "Amount owed is …" ve "You earned … frequent renter points" satırlarıyla biter.
- Movie sınıfında `public static final int CHILDRENS = 2; REGULAR = 0; NEW_RELEASE = 1;` sabitleri, `_title` ve `_priceCode` alanları vardır. Fiyat kodunun setter'ı vardır, çünkü yeni çıkan bir film bir süre sonra normal filme düşer; başlığın yalnız getter'ı vardır. Rental'ın kurucusu film ve gün sayısını alır, yalnız getter'ları vardır.

Ders sonrası not: Hoca kira kayıtlarının Java'nın bir bağlı listesinde (linked list) tutulduğunu söyledi (00:48:28). Ekrandaki kodda kayıtlar `Vector<Rental>` içinde tutuluyor ve `elements()` ile alınan `Enumeration` ile dolaşılıyor (ekrandan, 00:48:40, 00:50:00).

Ders sonrası not: Hoca `assertEquals` için iki nesnenin "aynı nesne" olması gerektiğini söyledi (00:25:48). `assertEquals` iki nesneyi `equals()` ile karşılaştırır; aynı referans olup olmadığını `assertSame` denetler. Örnekteki String karşılaştırmaları da içerik eşitliğine dayanır.

Ders sonrası not: Hoca JUnit 3'te test sınıfı adlarının da `test` ile başlamasının zorunlu olduğunu söyledi (00:24:02). JUnit 3'ün zorunlu tuttuğu, büyük olasılıkla, yalnız test metodu adlarının `test` ile başlamasıdır; sınıf adının `Test` ile başlaması bir adlandırma geleneğidir.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.